Silent Beacon — l'anomalie cachée dans la télémétrie

Starpwn CTF 2026 · DEF CON 34 · Aerospace Village

Catégorie : Space Operations **RÉSOLU — 12 août 2026**

Le satellite qui s'est tu

Un **CubeSat** — un satellite miniature de la taille d'une boîte de chaussures, le format standard des missions étudiantes et low-cost — espion s'est tu après une intrusion. La station sol a capté sa dernière rafale de **télémétrie** avant le silence. La télémétrie, c'est le bulletin de santé qu'un satellite envoie régulièrement vers la Terre : températures, tensions, modes de fonctionnement. Des centaines de valeurs banales... parmi lesquelles le satellite cachait quelque chose. Il faut le retrouver dans ce dump binaire de 5 835 octets.

Le challenge fournit trois fichiers, et c'est un point capital : deux d'entre eux sont de la **documentation**. `packet_ids.txt` est un extrait de manuel d'exploitation qui donne le marqueur de synchronisation et la liste des sous-systèmes. `telemetry_dictionary.json` décrit champ par champ le format de la télémétrie de housekeeping, avec les plages de valeurs autorisées. Dans ce genre de challenge, les fichiers annexes ne sont pas du décor : ils sont la moitié de la solution.

La Poste de l'espace (le paquet CCSDS)

Concept — La structure d'un paquet CCSDS. Un satellite ne parle pas à la Terre en phrases : il envoie des **paquets**, des petits blocs de données normalisés par le standard CCSDS 133.0-B (« Space Packet Protocol »). Le CCSDS est l'organisme qui standardise les communications spatiales — en gros, la Poste de l'espace : elle impose un format d'enveloppe pour que n'importe quelle station sol sache lire le courrier de n'importe quel satellite.

Le problème à résoudre par ce standard : le récepteur reçoit un **flux continu de bits**, sans ponctuation. Comment retrouver où commence un paquet ? Réponse : chaque paquet est précédé d'un **marqueur de synchronisation** (ASM, Attached Sync Marker) de 4 octets valant toujours `1A CF FC 1D`. C'est l'équivalent de la ligne de déchirure entre deux tickets de caisse : un motif fixe, choisi pour ne quasiment jamais apparaître par hasard dans les données. Le trouver dans le flux, c'est retrouver le début d'un paquet — et l'alignement de tous les octets qui suivent.

Viennent ensuite 6 octets d'**en-tête primaire** : un identifiant d'application (**APID**, sur 11 bits — le champ « quel sous-système parle », comme la mention « service comptabilité » sur une enveloppe interne), un **compteur de séquence** (pour remettre les paquets dans l'ordre, comme des pages numérotées), et la **longueur** du champ de données. Puis les données elles-mêmes.

Le fichier `packet_ids.txt` annonce trois APID : 50 (SYSLOG, du texte libre), 100 (HK_NOMINAL, le **housekeeping** — la télémétrie ménagère de routine : températures, tensions, mode de vol, charge fixe de 14 octets) et 200 (ADCS_STATUS, format non documenté — l'**ADCS** est le système de contrôle d'attitude, celui qui gère l'orientation du satellite). Le dictionnaire JSON ne détaille que l'APID 100 — c'est donc là qu'il faut chercher.

Deux leurres, une heure (~55 min perdues)

Deux leurres, une heure. Le premier (~30 min) : prendre le « bruit de ligne » de l'énoncé au pied de la lettre. Le dump a été passé au filtre, à la recherche d'une porteuse ou d'un motif périodique dans les octets de remplissage — il n'y a rien à filtrer. Le « bruit » est du simple **padding**, du remplissage entre les paquets, comme le papier bulle dans un carton. Le deuxième (~25 min) : l'APID 200 (ADCS_STATUS), dont le format non documenté fait penser que le secret s'y cache. Après rétro-ingénierie de sa structure interne champ par champ, le verdict tombe : des valeurs d'attitude plausibles et rien d'autre. Un leurre soigneux. Pendant ce temps, la documentation — relue enfin — désignait l'APID 100 depuis le début.

Cherchez la ligne de déchirure (étape 1 : l'ASM)

Plutôt que de lire le fichier séquentiellement (et de se perdre dans le remplissage), on cherche toutes les occurrences du marqueur `1A CF FC 1D`, et on parse l'en-tête CCSDS qui suit chacune. « Parser » = lire les octets selon le format documenté : 11 bits d'APID, puis le compteur, puis la longueur.

```
marqueurs ASM trouves dans capture.bin : 124
paquets parses                         : 124
remplissage ecarte                     : 2966 octets (51 % du fichier)
repartition par APID :
   APID 100 (HK_NOMINAL)  -&gt; 89 paquets   &lt;-- le housekeeping documente
   APID 200 (ADCS_STATUS) -&gt; 21 paquets   (leurre, format inconnu)
   APID  50 (SYSLOG)      -&gt; 14 paquets   (leurre, texte anodin)
```

Les 89 paquets de housekeeping portent des compteurs de séquence de 0 à 88, sans trou. On les trie par ce compteur : c'est l'ordre chronologique d'émission, celui dans lequel le message caché va se lire.

Le champ qui triche (étape 2 : la violation de règle)

L'idée clé — Un secret se trahit par une violation de règle déclarée. Le dictionnaire JSON dit que le champ `mode` (offset 12 du payload, c'est-à-dire le 13e octet des données) est un entier dont la plage valide est 0 à 7 : les huit modes de vol du satellite (SAFE, NOMINAL, SCIENCE...). C'est une **contrainte déclarée** : les données réelles doivent la respecter. Or si quelqu'un a caché un message en détournant ce champ, les valeurs injectées n'ont aucune raison de rester entre 0 et 7. Il suffit donc de chercher les paquets qui violent la règle.

Sur les 89 paquets, 60 valeurs de `mode` restent dans la bande autorisée. 29 en sortent nettement — et toutes tombent dans la plage 48–125, qui est exactement la zone des **caractères ASCII imprimables** (en ASCII, 48 = '0', 65 = 'A', 97 = 'a'). Autrement dit : le satellite a détourné un champ numérique pour y glisser une lettre par paquet. En lisant ces octets dans l'ordre du compteur de séquence :

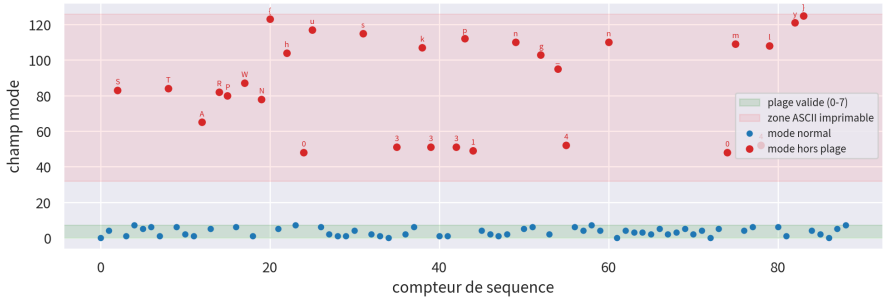


Figure 1 — Le champ mode des 89 paquets : les 60 valeurs sages restent dans la bande verte (0-7) ; les 29 intruses (rouge) tombent toutes dans la zone ASCII imprimable.

```
for p in housekeeping_tries_par_sequence:
    mode = p.payload[12]
    if mode &gt; 7:            # hors de la plage declaree par le dictionnaire
        flag += chr(mode)   # un caractere du flag, dans l'ordre

&gt;&gt;&gt; STARPWN{...}        # 29 caracteres, dans l'ordre des compteurs
```

`chr()` convertit un nombre en caractère ASCII : `chr(83)` donne 'S'. Le flag s'assemble lettre par lettre, paquet après paquet — sa lecture complète est gardée pour la fin, une fois la preuve établie.

Un témoin, c'est bien ; deux, c'est mieux (étape 3 : validation croisée)

Un flag qui « tombe » d'un seul critère laisse toujours un doute : et si les 29 valeurs hors plage étaient un hasard, une corruption du fichier ? Le format offre un second témoin totalement indépendant. Le champ `error_flags` (offset 13) contient huit **bits d'alarme** — huit petits drapeaux oui/non rangés dans un seul octet. Le bit 7 s'appelle ANOMALY : il est « armé » (mis à 1) quand le satellite détecte lui-même quelque chose d'anormal. Si les paquets qui transportent des lettres cachées sont aussi ceux que le satellite a marqués comme anormaux, les deux critères se désignent le même ensemble de paquets — et ce n'est plus une coïncidence.

```
paquets avec mode &gt; 7          : 29
dont le bit ANOMALY est arme   : 29 / 29
paquets 'sains' (mode&lt;=7) avec ANOMALY arme : 0
```

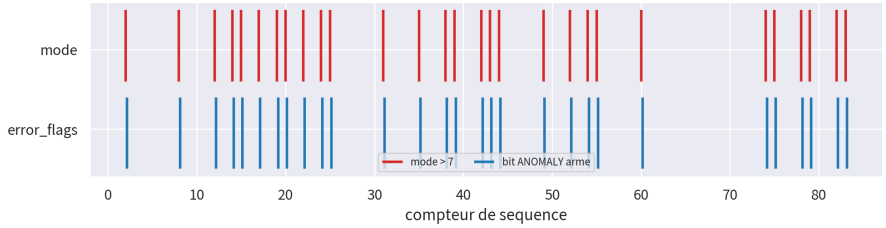


Figure 2 — Validation croisee : en rouge, les paquets dont mode > 7 ; en bleu pointille, ceux dont le bit ANOMALY est arme. Les deux courbes se superposent exactement.

Recouvrement parfait dans les deux sens : les 29 paquets porteurs sont exactement ceux marqués anormaux, et aucun paquet nominal n'est marqué à tort. Deux champs différents, deux règles différentes, une seule et même désignation. Le flag est **prouvé**, pas deviné. Cette discipline — un critère de validation qui aurait pu échouer — est ce qui sépare un solve solide d'une lecture chanceuse.

La morale de l'histoire

- **Les fichiers annexes sont souvent la solution** : le marqueur de synchro était écrit noir sur blanc dans `packet_ids.txt`, et la plage valide 0–7 dans le dictionnaire JSON. Trois lignes de code ont suffi là où une machinerie de resynchronisation aurait pu être écrite.
- **Un secret dans de la télémétrie se repère par les violations des règles déclarées** : le format lui-même dit ce qui est « normal », donc ce qui ne l'est pas saute aux yeux dès qu'on connaît la règle.
- **Un critère de validation doit pouvoir échouer, et deux valent mieux qu'un** : le bit ANOMALY aurait très bien pu ne pas coïncider avec le champ `mode` ; le fait qu'il coïncide parfaitement transforme une lecture plausible en preuve.

Reste à lire ce que les 29 paquets anormaux égrainaient depuis le début, lettre après lettre : `STARPWN{h0us3k33p1ng_4n0m4ly}` — validé sur la plateforme. Extraction confirmée par deux critères indépendants (mode hors plage et bit ANOMALY) désignant les 29 mêmes paquets. Le parser est dans le dépôt : `github.com/ShymusQuantum/starpwn-2026` (`silent_beacon/parse_hk.py`).